Starting with Django

Django is a web development framework that every Python developer should experience.

Kshitij Sobti
kshitij.sobti@thinkdigit.com

Django is a free open source framework for developing web sites. It offers unprecedented ease in defining complex data structures. It emphasises rapid development and the DRY principle (Don't Repeat Yourself). There are several advantages of Django. For example, the applications you create cleanly separate the model (the part that describes your data), the view (the part that generates the data to be displayed) and the template (the part that defines the visual appearance of your content).

Django lets you describe how your data is related in Python by defining model classes. For whatever kind of data structure you create for your web site, an administration panel is automatically created, and is usable out-of-the-box. The panel can be further customised for even better usability.

Installing Django

You need Python 2.7, not Python 3. Refer to bit.ly/python-install for directions on installing it on Windows.

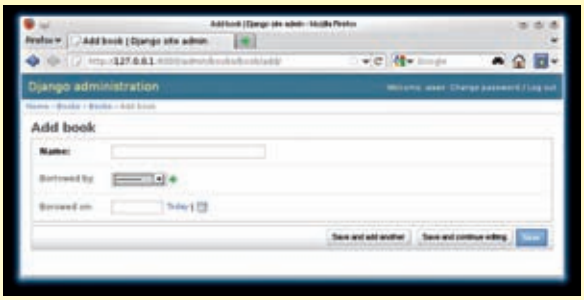
If you're running Linux, it's likely you already have Python and may be, even Django. In this case, install from your distro's repos – on openSUSE it's `sudo zypp install python-django`. Similarly, on Ubuntu it's `sudo apt-get install python-django`.

If these don't work, go to https://www.djangoproject.com/. Extract the archive to a known directory, and in the command line go to the directory in which you have extracted Django. Now run `sudo python setup.py install` on Linux or `python setup.py install` on Windows. Although you'll need an SQL server and a web server to serve your Django application in a production environment, we'll use the inbuilt SQLite database and inbuilt server for now.

Creating your Django site

We'll be making a simple application that lets you track books you've lent to others.

First, let's decide the structure of our data. We need to be able to store a list of books and persons, and associate a book with a person if it's lent to them. A person will have a name, a phone number and an email address so you can contact them. A book will have a name, an entry for the person who took it and the borrowing date. Now, let's move on to the next few steps.



Adding a Book

1. Create a site

Go to a folder where you wish to start this project in the command line and type the following:

```
python django-admin.py startproject myproject
```

This will create the basic structure of your web site inside the directory "myproject". Enter that directory.

Note: django-admin.py might not be in your path on Windows. You can find it at `C:\Python27\Scripts\django-admin.py` in which case use that path instead of just `django-admin.py`.

2. Create an app

You've created a web site, but it needs to have an application that contains a bulk of the code. A site may have multiple apps and an app might be used by multiple sites. You can create one by using the command:

```
python mange.py startapp books
```

3. Set up your database

Open the file called "settings.py" in your project folder in a text editor like Notepad++. Right in the beginning you'll find a configuration setting called DATABASES, under which you will find "default". Under the default database setting, change the ENGINE setting to `django.db.backends.sqlite3`, and the NAME to something like `myproject.db`. The NAME setting is just a path to the database.

4. Add your app to your site

In the same settings file, go down to the INSTALLED_APPS setting, and add `books` – the name of your application – right at the end.

5. Enable the admin panel

At this point, you might also go ahead and uncomment the line that says `django.contrib.admin` by removing the "#" in the beginning of that line. Close the settings.py file. Open the "urls.py" file in the same directory and uncomment the admin lines as instructed in the file. Close the file.

6. Create the models

Open the books directory in your project folder and open the models.py file for editing. Add the following to the file (it should already have `"from django.db import models"` else add that as well):

```
  class Person(models.Model):
      first_name = models.CharField(max_length=50)
      last_name = models.CharField(max_length=50, blank=True)
      phone = models.CharField(max_length=15, blank=True)
      email = models.EmailField(blank=True)
       def __unicode__(self):
              return self.first_name + " " + self.last_name
  class Book(models.Model):
      name = models.CharField(max_length=255)
      borrowed_by =
```